

ghost-rec

A private screen recorder for Windows 11 that leaves nothing on screen. No window, no tray icon, no notifications, no preview — ever. Controlled entirely by two hotkeys.

PLATFORM Windows 11, Python 3.10+	OUTPUT H.264 MP4, 30 fps, CRF 20
CONTROLS Ctrl+Shift+R · Ctrl+Shift+Q	NETWORK None. Local files only.

What it is

ghost-rec is a single Python script that records your screen to timestamped MP4 files while showing no indication that recording is happening. It runs under `pythonw.exe`, the windowless Python interpreter, so there is never a console window. It registers no tray icon, posts no notifications, and opens no preview. The screen looks exactly the same whether it is idle, recording, or off.

The design constraint was absolute: **the computer must not signal that recording is active**. Every design choice in the script serves that constraint first, then recording quality.

Contents

1. Quick start — install and first recording
2. Controls — the two hotkeys
3. Architecture — threads, pipeline, encoding
4. Configuration reference — every setting
5. Code structure — how the file is organised
6. Stealth model — what is visible, what is not
7. Privacy and data flow — where the video goes
8. Setup in detail — autostart and single-file exe
9. Verification status — what has been tested
10. Limitations and future options — audio, multi-monitor
11. Troubleshooting

Files in the project: `ghost_rec.py` (the recorder, ~230 lines), `requirements.txt` (five pip packages), `README.md` (short setup notes). This report is the full reference; the script runs standalone without it.

Quick start

From a fresh Windows 11 machine to a recording in under two minutes.

1 — Install Python

Install Python 3.10 or newer from python.org. During setup, tick “**Add python.exe to PATH**”. This also installs pythonw.exe, the windowless interpreter ghost-rec runs under.

2 — Install dependencies

Open a terminal in the project folder and run:

```
pip install -r requirements.txt
```

This installs five packages:

Package	Role
mss	Fast cross-platform screen capture (grabs monitor pixels)
numpy	Frame conversion: mss returns BGRA, the encoder needs RGB
imageio	Python-side video writer API
imageio-ffmpeg	Bundles a static ffmpeg binary — no system ffmpeg install needed
keyboard	Global hotkey registration (works when any app has focus)

3 — Run it

```
pythonw ghost_rec.py
```

Use pythonw, not python. The w variant never creates a console window. The script starts silently in the background and waits for a hotkey. Nothing appears on screen.

4 — Record

Press **Ctrl+Shift+R**. Recording starts immediately — no confirmation, no indicator. Press **Ctrl+Shift+R** again to stop. The MP4 finalises and lands in %USERPROFILE%\Videos\ghost\. Press **Ctrl+Shift+Q** to quit the recorder entirely.

How do you know it worked? Open videos\ghost\. The current session’s file — ghost_YYYY-MM-DD_HH-MM-SS.mp4 — grows in size while recording. That growing file and a two-line entry in ghost.log are the only confirmations the tool gives. Deliberately.

Controls

The entire interface is two global hotkeys. Both work regardless of which application has focus.

Hotkey	Action	What happens
Ctrl+Shift+R	Toggle recording	Idle → starts a new MP4. Recording → stops, finalises the file, returns to idle.
Ctrl+Shift+Q	Quit	If recording, finalises the current file first. Then unregisters hotkeys and exits. Nothing is left running.

Both keys are constants in the CONFIG block (see page 5) and can be changed to any combination the keyboard library accepts, e.g. "ctrl+alt+r".

Architecture

Three cooperating pieces: a capture loop, a per-session encoder, and a hotkey-driven state machine. Capture and encoding run on separate threads so a slow disk or a busy CPU never stalls the screen grab — and the screen grab never stalls the machine.



Capture thread — `_grab_loop()`

One daemon thread owns an `mss.mss()` context for the life of the process. When recording is active, it grabs the configured monitor, converts each frame from BGRA to RGB with a numpy slice reversal (`[ :, :, 2::-1]`), and pushes it into the queue. When idle, it drains any stale frames and sleeps 50 ms. The loop is wrapped in a broad exception handler: a capture error is written to the log and never kills the hotkey thread, so the quit hotkey keeps working even if capture dies.

Frame queue — drop rather than lag

The queue holds at most 90 frames (3 seconds at 30 fps). If the encoder falls behind, `put()` times out after 50 ms and the frame is discarded. The machine never stutters waiting for the recorder; the video may drop frames under heavy load instead. For a privacy recorder this is the right trade: system responsiveness is invisible, dropped frames are not.

Writer thread — `_write_loop()`

Each recording session gets its own writer thread. It opens an `imageio` writer backed by the bundled static `ffmpeg` from `imageio-ffmpeg` — no system `ffmpeg` installation is required. The writer drains the queue and appends frames until the session-stop event fires and the queue empties, then closes the file so the MP4 is playable. Encoder settings: `libx264` codec, CRF 20, `veryfast` preset, `yuv420p` pixel format for maximum player compatibility, and `macro_block_size=1` so odd screen resolutions are not padded.

State machine — `toggle()` / `quit_app()`

A single lock (`_state_lock`) serialises hotkey callbacks. `toggle()` flips the `_recording` flag and either starts a writer thread (generating the timestamped filename) or signals it to finish and joins it (15-second timeout). `quit_app()` finalises any in-progress recording before setting the quit event, so `Ctrl+Shift+Q` mid-recording never corrupts the file. The main thread blocks on the quit event; hotkeys are unregistered on exit.

Configuration reference

Every behaviour is a constant in the CONFIG block at the top of `ghost_rec.py`. Edit, save, relaunch — no other file changes.

```
# ----- CONFIG -----
FPS = 30
MONITOR_INDEX = 1          # 1 = primary monitor, 0 = all monitors at once
OUTPUT_DIR = Path.home() / "Videos" / "ghost"
HOTKEY_TOGGLE = "ctrl+shift+r"
HOTKEY_QUIT = "ctrl+shift+q"
CRF = 20                   # video quality: 18 = near-lossless/big, 23 = small
SINGLE_INSTANCE_PORT = 53917 # localhost only; change if it clashes
# -----
```

Setting	Default	What it controls
FPS	30	Target frame rate for capture and the MP4's declared rate. Lower (15–24) for smaller files on mostly-static screens.
MONITOR_INDEX	1	Which monitor to grab. 1 = primary. 0 = all monitors stitched into one frame. 2, 3 = additional displays. Out-of-range values fall back to 1.
OUTPUT_DIR	~/Videos/ghost	Where MP4s and <code>ghost.log</code> are written. Created automatically on launch.
HOTKEY_TOGGLE	ctrl+shift+r	Start/stop recording. Any string the keyboard library accepts.
HOTKEY_QUIT	ctrl+shift+q	Quit the recorder entirely.
CRF	20	x264 quality. 18 = near-lossless, large files. 20 = default balance. 23–28 = visibly softer, much smaller. 0 = lossless (very large).
SINGLE_INSTANCE_PORT	53917	Localhost UDP port for the single-instance guard. Change only if another program uses the same port.

Code structure

The script is a single file of about 230 lines. Four pieces, each small enough to audit at a glance:

Piece	Lines	Job
CONFIG block	Top of file	All tunables (above). Nothing else in the file needs editing for normal use.
<code>_grab_loop()</code>	Capture thread	mss grab → BGRA-to-RGB conversion → bounded queue push. Drops frames on backpressure. Drains the queue while idle.
<code>_write_loop()</code>	Per-session thread	Opens the imageio/ffmpeg writer, drains the queue into libx264, closes the MP4 cleanly on stop.
<code>_start_session()</code> / <code>_end_session()</code> , <code>toggle()</code> , <code>quit_app()</code>	Session control	Timestamped filename generation, thread lifecycle, lock-serialised hotkey handling.
<code>_already_running() + main()</code>	Entry point	Single-instance guard, output-dir creation, hotkey registration, block-until-quit.
<code>_log()</code>	Throughout	Appends one timestamped line per event (start, stop, saved file, errors) to <code>ghost.log</code> . Silent on failure — logging never crashes the recorder.

Stealth model

The requirement was blunt: the computer must show no indication that recording is happening. This section is the honest accounting of what that means in practice — what is hidden and what cannot be.

What never appears

Surface	Status
Window or console	Never. Runs under <code>pythonw.exe</code> ; no console is ever allocated.
Taskbar / Alt-Tab entry	Never. No window means no taskbar presence.
System tray icon	Never. The script creates no tray presence (no <code>pystray</code> , no notify icon).
Notifications / toasts	Never. No notification API is called on start, stop, or error.
Recording indicator / overlay	Never. Nothing is drawn to the screen at any point.
Cursor or screen change on toggle	Never. The hotkey is consumed by the keyboard hook; the foreground app sees no side effect.
Sound	Never. No audio output of any kind.

What does exist (the honest traces)

Trace	Where	Notes
Process	Task Manager → Details	<code>pythonw.exe</code> is listed like any Python process. Its command line shows the script path. Renaming via the PyInstaller exe (page 10) changes the display name.
Video file	<code>%USERPROFILE%\Videos\ghost\</code>	The in-progress MP4 grows while recording. This is also the intended way to confirm recording is active.
Log file	<code>Videos\ghost\ghost.log</code>	Tiny text file: launch, recording start/stop, saved filename, errors. Timestamps only — no frame content, no metadata about what was on screen.
CPU / disk activity	Task Manager → Performance	Capture + x264 encode costs CPU (a few percent at 1080p30; more at 4K or CRF 18) and writes continuously while recording. Visible in aggregate graphs, not surfaced on the desktop.

Single-instance guard. Launching `ghost-rec` twice does not start a second recorder. On launch the script binds a UDP socket to `127.0.0.1:53917`. If the bind fails, another copy is already running and the new process exits silently — no error, no dialog. The socket is held for the life of the process and released by the OS on exit, so there are never stale lockfiles to clean up after a crash.

Privacy and data flow

Zero network connections. The script imports no HTTP client, opens no outbound socket, and phones nothing home. The single localhost UDP bind above is the only socket in the program and it carries no data — it exists purely as a mutex. All video is written to the local output folder and never leaves the machine unless you move it.

There is no account, no telemetry, no analytics, and no update check. Dependencies are installed once via pip; at runtime the recorder is fully offline.

Setup in detail

Prerequisites

- Windows 11 (Windows 10 also works; the hotkey and capture stack is not version-specific).
- Python 3.10+ from python.org with “**Add python.exe to PATH**” ticked. This provides both `python.exe` and `pythonw.exe`.
- ~50 MB free disk for the Python packages and bundled ffmpeg; video files are additional.

Install

```
cd path\to\screen-recorder
pip install -r requirements.txt
pythonw ghost_rec.py
```

The output folder `%USERPROFILE%\Videos\ghost\` is created on first launch. No admin rights are required for normal use. One exception: global hotkeys cannot intercept keystrokes directed at an elevated (Run as Administrator) window unless ghost-rec itself runs elevated. For desktop recording this rarely matters.

Output files

File	Example	Contents
Video	ghost_2026-10-05_22-31-04.mp4	One file per session, named by start time. H.264, yuv420p — plays in Windows Media Player, VLC, browsers, and editors.
Log	ghost.log	Append-only event log: ghost-rec started, recording started -> ghost_...mp4, saved ghost_...mp4, ghost-rec stopped, plus any errors. Aids debugging without touching the screen.

Auto-start at login (optional)

To have the recorder waiting at every login:

1. Press `Win+R`, type `shell:startup`, press Enter. The Startup folder opens.
2. Create a shortcut in it. Target: `pythonw.exe "c:\path\to\ghost_rec.py"` (adjust the path). Set “Start in” to the script’s folder.
3. On next login the recorder starts silently. Quit it any time with `Ctrl+Shift+Q`; remove the shortcut to disable autostart.

Distribution options

Single-file exe (optional)

To run without a visible Python installation — and to change the process name from `pythonw.exe`:

```
pip install pyinstaller
pyinstaller --noconsole --onefile ghost_rec.py
```

`--noconsole` matches the `pythonw` behaviour (no window). `--onefile` produces a single `ghost_rec.exe` in `dist\`. The bundled `ffmpeg` is extracted to a temp folder at runtime by `PyInstaller`; the first launch after a reboot may take a few seconds longer. The exe can be renamed freely.

Confirming it is recording

By design there is no on-screen confirmation. Use one of:

- **Output folder:** the newest `ghost_*.mp4` grows in size while recording; its size stops changing after `Ctrl+Shift+R` stops the session, and the file becomes playable once finalised.
- **Log tail:** `ghost.log` gains a recording started line on start and a saved line on stop.
- **Task Manager** → **Details:** `pythonw.exe` (or the exe name) is present while the recorder is resident, recording or idle.

Verification status

Honest accounting of what has and has not been exercised:

Check	Status
Syntax	Parsed clean — <code>ghost_rec.py</code> compiles with no errors.
Imports	All five dependencies (<code>mss</code> , <code>keyboard</code> , <code>imageio</code> , <code>imageio-ffmpeg</code> , <code>numpy</code>) import successfully in a Linux test environment with the pinned requirements.
Logic review	Thread lifecycle, queue backpressure, session start/stop, and the single-instance guard read through against the library APIs (<code>mss</code> monitor indexing, <code>imageio</code> <code>ffmpeg</code> writer parameters, <code>keyboard</code> hotkey syntax).
Full runtime test	Not yet run on Windows. Screen capture, global hotkeys, and the bundled <code>ffmpeg</code> binary are Windows-runtime behaviours that cannot be exercised from the Linux build environment. First-run validation on the target Windows 11 machine is still required: launch, one short recording, playback check, quit.

Nothing in this report claims a completed Windows recording until that check is done. The expected first-run failure modes are mundane — a missing `PATH` entry for `pythonw`, or an antivirus flagging the unsigned `PyInstaller.exe` — and both are covered in Troubleshooting below.

Limitations and future options

No audio (current)

`ghost-rec` records video only. There is no microphone or system-audio track. This is the single largest functional gap versus general-purpose recorders.

How audio would be added: capture system audio via WASAPI loopback (e.g. `PyAudioWPatch` or `soundcard`) and/or microphone via `sounddevice`, on a third thread feeding a parallel sample queue; then either mux at write time by piping audio into the same `ffmpeg` process (`imageio` supports raw `ffmpeg` pipes) or write a WAV sidecar and mux with a second `ffmpeg` pass on session stop. Loopback keeps the “no indication” property — Windows shows no mic indicator for loopback capture, whereas live microphone capture triggers the taskbar microphone icon while the mic is hot. That trade-off should be a `CONFIG` switch, defaulting to off.

Multi-monitor, limits, and what is next

Multi-monitor notes

- `MONITOR_INDEX = 1` records the primary display only — the cursor and windows on other monitors are not captured.
- `MONITOR_INDEX = 0` captures all monitors as one stitched frame. Resolution (and therefore file size and CPU cost) scales with total desktop area; mixed-DPI setups may show scaling artefacts from mss.
- 2, 3, ... select individual secondary monitors. An out-of-range index falls back to the primary rather than crashing.
- Per-monitor simultaneous recording (separate file per display) is not implemented; it would mean one capture and writer thread pair per monitor.

Other limits

- **Frame drops under load:** by design. Heavy CPU contention drops frames instead of stuttering the machine (3-second queue, 50 ms push timeout).
- **Crash safety:** if the process is killed mid-recording, the in-progress MP4 may lack its index (moov atom) and refuse to play. It is usually recoverable with `ffmpeg -i broken.mp4 -c copy fixed.mp4`-style remux tools, but it is not automatic.
- **Elevated windows:** hotkeys do not fire when an administrator window has focus unless ghost-rec also runs elevated (Windows UIPI). The capture itself still records that window's contents.
- **Fullscreen exclusive games:** mss captures the desktop compositor; true exclusive-fullscreen titles may record as black. Borderless fullscreen works. This is a limitation of desktop duplication generally, not of this script specifically.

Reasonable next features (not built)

- Optional system-audio loopback track (above), default off.
- Segmented recording: roll to a new file every N minutes for very long sessions.
- Timed auto-stop (e.g. stop after 2 hours) as a CONFIG option.
- Disk-space guard: stop cleanly when the output volume drops below a threshold.

Troubleshooting

Symptom	Likely cause and fix
Nothing happens on launch; no log file	<code>pythonw</code> is swallowing an early error by design (no console). Run once with <code>python ghost_rec.py</code> (with console) to see the traceback, fix, then return to <code>pythonw</code> . Most common: dependencies not installed for this Python, or Python not on PATH.
Second launch “does nothing”	Working as intended — the single-instance guard exited silently because a copy is already running. Quit it with Ctrl+Shift+Q, or end <code>pythonw.exe</code> in Task Manager, before relaunching.
Hotkeys do nothing	An elevated window has focus (see Limitations), or another program has claimed the same hotkey. Change <code>HOTKEY_TOGGLE/HOTKEY_QUIT</code> in <code>CONFIG</code> and relaunch.
Video is a black screen	Exclusive-fullscreen app (see Limitations), or protected/DRM content that the compositor blanks. Test with a normal desktop window first.
File will not play	Process was killed mid-write. Try a remux (<code>ffmpeg -i in.mp4 -c copy out.mp4</code>) or a player with damaged-file recovery such as VLC.
Antivirus flags the exe	Unsigned PyInstaller onefile builds commonly trigger heuristic detections. Signing the exe, building on the target machine, or running the plain <code>.py</code> under <code>pythonw</code> avoids it.